Chapter 7. MCP(Model Context Protocol) 이해 및 활용

학습 목표

이번 장에서는 MCP(Model Context Protocol)의 개념과 역할을 이해하고, Google ADK에서 MCP Server를 연결하여 외부 Tool을 사용하는 방법을 학습합니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- MCP의 개념 이해
- MCP Client와 MCP Server 구조 이해
- Google ADK에서 MCP Server 연동
- MCP를 활용한 AI Agent 구현

이 장의 용어

용어	영문 · 약어 풀이	설명
MCP	Model Context Protocol	AI 모델과 외부 시스템이 Tool·데이터를 주고받는 방식을 표준화한 규약
프로토콜	Protocol, 통신 규약	서로 통신하기 위해 미리 정해 둔 규칙과 형식. HTTP, MCP 모두 프로토콜
Host	호스트	AI Agent가 실행되는 애플리케이션. adk web, Claude Desktop 등
Client / Server	클라이언트 / 서버	요청을 보내는 쪽이 클라이언트, 요청을 받아 처리하는 쪽이 서버. MCP Client는 Host 안에 있고 MCP Server는 실제 기능을 제공
Resource	리소스	MCP Server가 제공하는 읽기용 데이터(파일, 문서 등). Tool은 실행, Resource는 조회
stdio	Standard Input/Output, 표준 입출력	같은 PC 안에서 프로그램끼리 입력·출력 통로로 데이터를 주고받는 방식. 로컬 MCP 서버의 기본 전송 방식
Streamable HTTP	스트리밍 가능 HTTP	HTTP 위에서 요청을 보내고 응답을 생성되는 대로 조금씩 받는 MCP의 원격 전송 방식
프로세스	Process	실행 중인 프로그램 하나. MCP 서버는 Agent와 별도의 프로세스로 동작

용어	영문 · 약어 풀이	설명
샌드박스	Sandbox	프로그램이 접근할 수 있도록 허용한 격리된 영역. 실습에서는 `sandbox` 폴더 밖의 파일에 접근하지 못하게 제한
Node.js / npm / npx	Node Package Manager / Node Package Execute	Node.js는 JavaScript를 PC에서 실행하는 환경, npm은 그 패키지 설치 도구, npx는 설치 없이 패키지를 내려받아 바로 실행하는 명령. Python의 pip에 해당
FastMCP	—	Python 함수에 `@mcp.tool()`만 붙이면 MCP 서버가 되도록 도와주는 라이브러리
데코레이터	Decorator	`@이름` 형태로 함수 위에 붙여 그 함수에 기능을 더하는 Python 문법. `@mcp.tool()`이 함수를 Tool로 등록
인증	Authentication	요청을 보낸 사람이 누구인지 확인하는 절차. 원격 MCP 서버는 헤더의 토큰으로 확인
엔드포인트	Endpoint	원격 서버에서 요청을 받는 URL 경로. `http://127.0.0.1:8080/mcp`의 `/mcp`

1. MCP(Model Context Protocol)란?

1.1. MCP의 등장 배경

AI Agent는 단순히 질문에 답하는 수준을 넘어 다양한 외부 시스템과 연동하여 작업을 수행합니다.

예를 들어 AI Agent는 다음과 같은 작업을 수행할 수 있습니다.

- 현재 날씨 조회
- 데이터베이스 조회
- GitHub 저장소 검색
- Slack 메시지 전송
- Notion 문서 조회
- 파일 읽기 및 저장
- 이메일 발송

지금까지 우리는 이러한 기능을 **Tool**로 직접 구현했습니다.

예를 들어 날씨 정보를 조회하기 위해서는

```python
def get_weather(city: str):
    ...
```

와 같은 Tool을 만들고, Agent에 등록하여 사용했습니다.

하지만 프로젝트 규모가 커질수록 다음과 같은 문제가 발생합니다.

- Tool의 개수가 계속 증가한다.
- 외부 서비스마다 연동 방식이 다르다.
- 프로젝트마다 동일한 Tool를 반복해서 구현해야 한다.
- 다른 개발자가 만든 Tool를 재사용하기 어렵다.

즉, **AI Agent와 외부 시스템을 연결하는 공통된 표준이 필요하게 되었습니다.**

이러한 문제를 해결하기 위해 등장한 것이 **MCP(Model Context Protocol)** 입니다.

1.2. MCP(Model Context Protocol)이란?

MCP(Model Context Protocol)는 AI 모델과 외부 시스템이 **표준화된 방식으로 통신할 수 있도록 정의한 프로토콜(Protocol, 서로 통신하기 위해 미리 정해 둔 규칙과 형식)**입니다.

쉽게 말해,

> **AI Agent가 다양한 외부 기능을 동일한 방식으로 사용할 수 있도록 만든 공통 인터페이스입니다.**

예를 들어

```
GitHub

↓

GitHub Tool
```

```
Slack

↓

Slack Tool
```

```
Database

↓

Database Tool
```

처럼 각각 다른 방식으로 연결하던 것을

```
AI Agent

↓

MCP

↓

GitHub

Slack

Database

File System
```

와 같이 하나의 표준 방식으로 연결할 수 있습니다.

즉, AI Agent는 MCP만 이해하면 다양한 서비스를 사용할 수 있습니다.

1.3. 왜 MCP가 필요한가?

예를 들어 여러분이 회사에서 AI Agent를 개발한다고 가정해 보겠습니다.

다음과 같은 기능이 필요합니다.

- GitHub Issue 조회
- Jira 티켓 조회
- Slack 메시지 전송
- Google Drive 문서 검색
- 데이터베이스 조회

Tool 방식으로 구현한다면

```
GitHub Tool
```

```
Slack Tool

Jira Tool

Drive Tool

DB Tool
```

모든 Tool를 직접 개발해야 합니다.

또한 프로젝트가 바뀌면 다시 구현해야 할 수도 있습니다.

반면 MCP를 사용하면

```
GitHub MCP Server

Slack MCP Server

Jira MCP Server

Google Drive MCP Server

Database MCP Server
```

를 그대로 연결하여 사용할 수 있습니다.

즉, 이미 만들어진 MCP Server를 재사용할 수 있습니다.

1.4. AI Agent와 MCP의 관계

지금까지 학습한 Google ADK 구조를 다시 살펴보겠습니다.

```
사용자

↓

Agent

↓

Tool

↓

외부 API
```

Tool가 외부 API를 호출하는 역할을 수행했습니다.

MCP를 사용하면 구조가 다음과 같이 변경됩니다.

```
사용자

↓

Agent

↓

MCP Client

↓

MCP Server

↓

외부 시스템
```

여기서

- Agent는 사용자의 요청을 이해합니다.
- MCP Client는 MCP Server와 통신합니다.
- MCP Server는 실제 기능을 수행합니다.

즉, Agent는 MCP Server가 어떤 언어로 구현되었는지,

어떤 API를 사용하는지 알 필요가 없습니다.

표준 방식으로 요청만 보내면 됩니다.

1.5. Tool Calling과 MCP의 차이

앞에서 학습한 Tool Calling과 MCP는 목적은 비슷하지만 구현 방식이 다릅니다.

Tool Calling

```
Agent

↓

Python Tool

↓

REST API
```

- Tool를 직접 구현해야 합니다.

- 프로젝트마다 Tool을 다시 작성할 수 있습니다.
- 개발자가 Tool을 관리합니다.

MCP

```
Agent

↓

MCP Client

↓

MCP Server

↓

REST API
```

- 표준 프로토콜을 사용합니다.
- 다양한 MCP Server를 재사용할 수 있습니다.
- 여러 Agent에서 동일한 MCP Server를 사용할 수 있습니다.

1.6. MCP의 장점

MCP를 사용하면 다음과 같은 장점이 있습니다.

표준화(Standardization)

모든 외부 기능을 동일한 방식으로 사용할 수 있습니다.

재사용성(Reusability)

한 번 만든 MCP Server를 여러 프로젝트에서 사용할 수 있습니다.

확장성(Scalability)

새로운 기능이 필요하면 새로운 MCP Server만 추가하면 됩니다.

기존 Agent는 수정할 필요가 거의 없습니다.

유지보수성(Maintainability)

GitHub 기능을 수정하더라도

GitHub MCP Server만 수정하면 됩니다.

Agent는 변경되지 않습니다.

언어 독립성(Language Agnostic)

MCP Server는 Python뿐 아니라

- Java
- Go
- Node.js
- Rust

등 다양한 언어로 구현할 수 있습니다.

Agent는 구현 언어를 신경 쓰지 않고 사용할 수 있습니다.

1.7. MCP 활용 사례

최근에는 다양한 MCP Server가 공개되어 있습니다.

대표적인 예는 다음과 같습니다.

MCP Server	기능
File System	파일 읽기 및 저장
GitHub	저장소 및 Issue 조회
Slack	메시지 조회 및 전송
SQLite	데이터베이스 조회
PostgreSQL	SQL 실행
Notion	문서 조회
Google Drive	파일 검색
Web Search	웹 검색

이처럼 이미 구현된 MCP Server를 활용하면 개발 시간을 크게 줄일 수 있습니다.

정리

이번 절에서는 MCP(Model Context Protocol)의 개념과 필요성을 학습했습니다.

개념	설명
MCP	AI Agent와 외부 시스템을 연결하는 표준 프로토콜

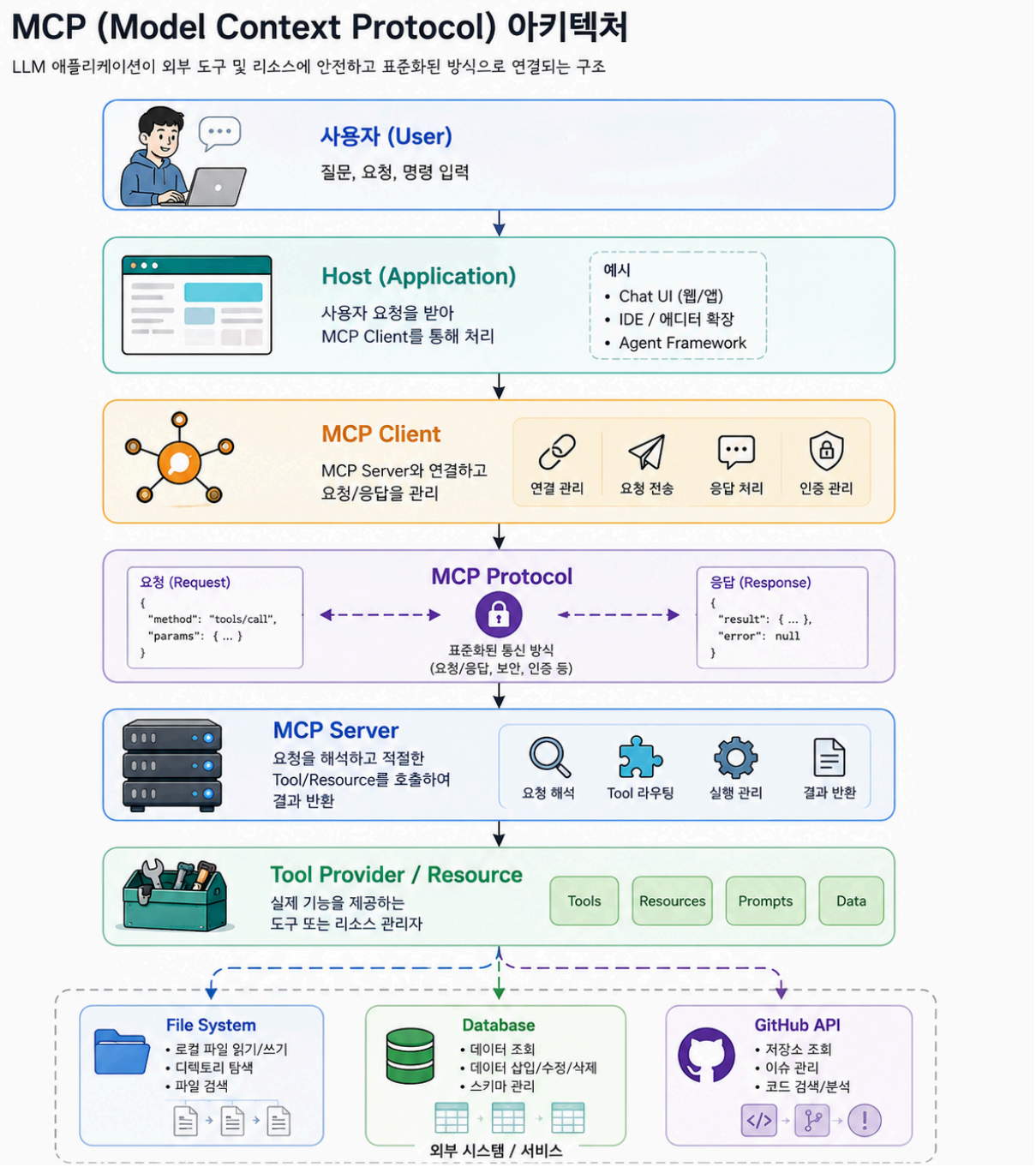
개념	설명
등장 배경	Tool 재사용성과 표준화 필요
역할	AI와 외부 서비스 간의 공통 인터페이스 제공
장점	표준화, 재사용성, 확장성, 유지보수성
Google ADK	MCP Server를 연결하여 다양한 기능 활용

2. MCP 아키텍처 이해

2.1. MCP 아키텍처란?

MCP는 단순히 하나의 라이브러리가 아니라 여러 구성 요소가 함께 동작하는 구조를 가지고 있습니다.

전체 구조는 다음과 같습니다.



Host는 AI Agent가 실행되는 프로그램이며,

MCP Client와 MCP Server는 MCP 프로토콜을 이용하여 서로 통신합니다.

2.2. Host(Application)

Host는 AI Agent가 실제로 실행되는 애플리케이션입니다.

쉽게 말하면 사용자가 직접 사용하는 프로그램입니다.

예를 들어 다음과 같은 프로그램이 Host가 될 수 있습니다.

- Google ADK Agent
- ChatGPT Desktop
- Claude Desktop
- VS Code Extension
- 사내 AI Assistant
- 고객 상담 AI

Host의 역할은 다음과 같습니다.

- 사용자 입력 받기
- AI Agent 실행
- MCP Client 생성
- MCP Server 호출
- 최종 응답 출력

즉, Host는 **사용자와 AI Agent를 연결하는 시작점**입니다.

2.3. MCP Client

MCP Client는 Host 내부에서 실행되며 MCP Server와 통신하는 역할을 담당합니다.

쉽게 말하면

> **MCP Client는 AI Agent의 대리인입니다.**

Agent가

```
현재 서울 날씨를 알려줘.
```

라고 요청하면

MCP Client는

```
Weather MCP Server에게
서울의 현재 날씨를 요청
```

하는 역할을 수행합니다.

MCP Client의 주요 기능은 다음과 같습니다.

- MCP Server 연결
- Tool 목록 조회
- Tool 실행 요청
- Resource 조회
- Prompt 조회
- 결과 수신

AI Agent는 MCP Server와 직접 통신하지 않고 반드시 MCP Client를 통해 요청을 전달합니다.

2.4. MCP Server

MCP Server는 실제 기능을 제공하는 프로그램입니다.

예를 들어

- 파일 읽기
- 데이터베이스 조회
- GitHub 검색
- Slack 메시지 전송

등의 기능을 제공합니다.

예를 들어 File System MCP Server는

```
파일 읽기

파일 저장

폴더 조회

파일 삭제
```

와 같은 기능을 제공합니다.

GitHub MCP Server는

```
Repository 조회

Issue 조회

Pull Request 조회
```

와 같은 기능을 제공합니다.

즉,

> **MCP Server는 Tool와 Resource를 외부에 제공하는 서버입니다.** 여기서 Resource(리소스)는 파일이나 문서처럼 읽어 갈 수 있도록 제공되는 데이터를 뜻합니다. Tool은 실행하는 것, Resource는 읽는 것입니다.

2.5. Tool Provider

MCP Server 내부에는 여러 기능이 존재합니다.

이 기능들을 **Tool Provider**라고 생각하면 이해하기 쉽습니다.

예를 들어 File System MCP Server에는

```
read_file()

write_file()

list_directory()

create_directory()
```

와 같은 Tool가 포함되어 있습니다.

Database MCP Server에는

```
execute_query()

insert()

update()

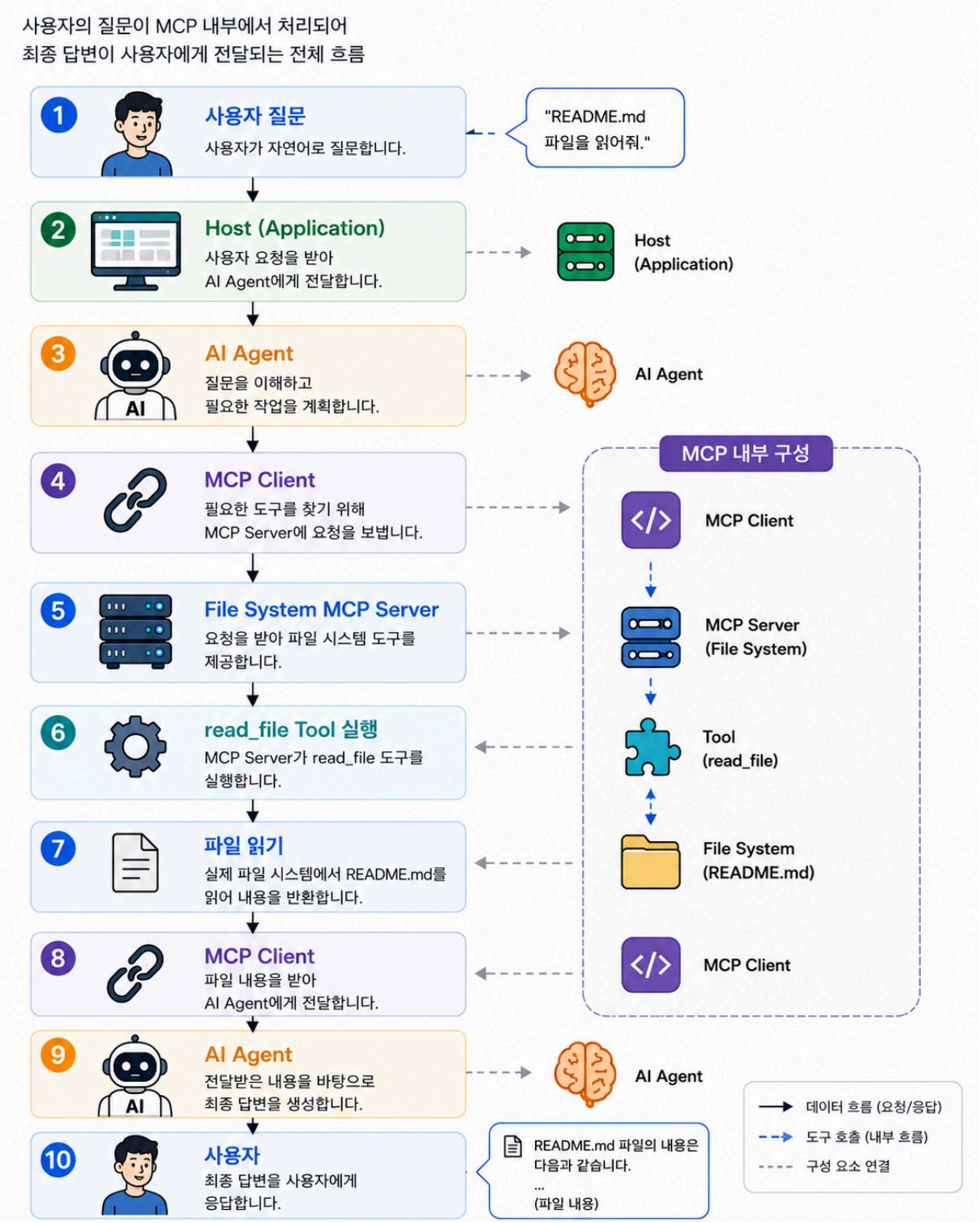
delete()
```

와 같은 Tool가 존재합니다.

AI Agent는 이러한 Tool를 직접 구현하지 않고 MCP Server를 통해 사용할 수 있습니다.

2.6. MCP의 데이터 흐름

사용자가 아래와 같이 질문하면 MCP 내부에서는 다음과 같은 흐름으로 데이터가 이동합니다.



2.7. Google ADK에서의 MCP 구조

Google ADK에서는 다음과 같은 구조로 MCP를 사용합니다.

```
                Google ADK
                    |
```

```
                  Root Agent
                      |
                 MCP Client
                      |
──────────── MCP Protocol ────────────
                      |
                 MCP Server
                      |
             Tool / Resource
                      |
          GitHub · Database · File
```

Google ADK는 MCP Client를 통해 다양한 MCP Server와 연결하여 외부 기능을 사용할 수 있습니다.

정리

이번 절에서는 MCP의 전체 아키텍처와 각 구성 요소의 역할을 학습했습니다.

구성 요소	역할
Host(Application)	AI Agent가 실행되는 애플리케이션
MCP Client	MCP Server와 통신하는 클라이언트
MCP Server	Tool와 Resource를 제공하는 서버
Tool Provider	실제 기능을 수행하는 Tool 집합
MCP Protocol	Client와 Server 간의 표준 통신 방식

---

3. Google ADK에서 MCP Server 연동 실습

실습 개요

이번 실습에서는 앞서 배운 MCP 아키텍처(Host–Client–Server)를 직접 구축해봅니다. 기존 MCP 서버를 가져다 쓰는 것부터, 나만의 MCP 서버를 만들어 연동하는 것까지 순서대로 진행합니다.

모든 실습은 Chapter 2부터 사용해 온 `my-adk` 프로젝트 안에서 진행하며, 새 프로젝트나 가상 환경을 만들지 않습니다.

단계	내용
3.1	실습 환경 준비
3.2	실습 7-1 — 기존 MCP 서버(Filesystem) 연동
3.3	실습 7-2 — 나만의 MCP 서버 만들기 (FastMCP)

단계	내용
3.4	실습 7-3 — 내 MCP 서버를 ADK 에이전트에 연동
3.5	실습 7-4 — 원격/인증 MCP 서버 연동 (선택)
3.6	트러블슈팅 가이드
3.7	실습 정리 및 점검

3.1 실습 환경 준비

체크리스트 — 실습을 시작하기 전 아래 항목을 모두 확인합니다.

- [ ] `my-adk` 프로젝트를 VS Code에서 열고 가상환경이 활성화되어 있음 (`(venv)` 표시)
- [ ] `.env`에 `GOOGLE_API_KEY`가 설정되어 있음 (Chapter 2에서 완료)
- [ ] Node.js / npx 설치 (실습 7-1의 커뮤니티 MCP 서버 실행용)

Node.js 설치

실습 7-1에서 사용하는 Filesystem MCP 서버는 Node.js(JavaScript를 PC에서 실행하는 환경)로 만들어진 패키지이므로 `npx` 명령(Node Package Execute, 설치 없이 패키지를 내려받아 바로 실행)이 필요합니다.

1. nodejs.org 접속
2. **Node.js 24** 버전 다운로드 후 기본 옵션으로 설치 (2일차 n8n이 Node.js 24 이상을 요구하므로 같은 버전을 설치합니다)
3. VS Code를 **완전히 종료한 뒤 다시 실행**하고, 새 터미널(명령 프롬프트)에서 아래 명령으로 확인

```
node -v
npx -v
```

버전 숫자가 출력되면 정상입니다. `'node'은(는) 내부 또는 외부 명령... 이 아닙니다`가 나오면 VS Code가 Node.js 설치 전의 환경 정보를 그대로 쓰고 있는 것입니다. 터미널만 새로 열어서는 반영되지 않으므로 VS Code 창을 모두 닫고 다시 실행합니다.

추가 패키지 설치

가상환경이 활성화된 터미널에서 다음을 실행합니다.

```
pip install mcp fastmcp
```

`requirements.txt`에도 추가해 둡니다.

```
google-adk
python-dotenv
requests
mcp
fastmcp
```

> **확인:** `python -c "import google.adk, fastmcp; print('OK')"` 실행 시 오류 없이 `OK`가 출력되면 준비 완료입니다.

이번 장에서 추가되는 폴더

```
my-adk/
├── .env
├── agent/                    # 기존 Agent
├── helpdesk/                 # Chapter 6 Agent
├── tools/
│
├── sandbox/                  # 실습 7-1: MCP 서버가 접근할 파일 폴더
│   └── sample.txt
├── filesystem_agent/         # 실습 7-1
├── my_mcp_server/            # 실습 7-2: 직접 만드는 MCP 서버
│   └── server.py
├── lab_tools_agent/          # 실습 7-3
└── remote_mcp_agent/         # 실습 7-4 (선택)
```

3.2 실습 7-1 — 기존 MCP 서버(Filesystem) 연동

목표: 이미 만들어진 공식 MCP 서버를 ADK 에이전트에 붙여, "가져다 쓰는" 가장 기본적인 흐름을 익힙니다.

Step 1. 실습용 폴더와 샘플 파일 생성

샌드박스(Sandbox)는 프로그램이 접근할 수 있도록 허용한 격리된 영역을 뜻합니다. MCP 서버가 PC의 모든 파일을 다루지 못하도록 이 폴더 안으로 범위를 제한합니다.

VS Code의 탐색기에서 `my-adk` 바로 아래에 `sandbox` 폴더를 만들고, 그 안에 `sample.txt` 파일을 생성해 아래 내용을 입력하고 저장합니다.

```
MCP 실습용 샘플 파일입니다.
```

> **참고**
> 명령 프롬프트에서 `echo ... > sample.txt`로 파일을 만들면 한글이 Windows 기본 인코딩(CP949)으로 저장되어, UTF-8로 읽는 MCP 서버에서 글자가 깨집니다. VS Code에서

> 직접 파일을 만들면 UTF-8로 저장되므로 안전합니다(VS Code 오른쪽 아래 상태 표시줄에 `UTF-8`로 표시됨).

Step 2. 에이전트 폴더 구조 만들기

```
my-adk/
├── sandbox/
│   └── sample.txt
└── filesystem_agent/
    ├── __init__.py
    └── agent.py
```

Step 3. `agent.py` 작성

```
# filesystem_agent/agent.py
import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import Std
ioConnectionParams
from mcp import StdioServerParameters

# my-adk/sandbox 폴더의 절대 경로
SANDBOX_PATH = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file_
_))), "sandbox"
)

root_agent = LlmAgent(
    model="gemini-3.1-flash-lite",
    name="filesystem_assistant_agent",
    instruction=(
        "사용자의 파일을 조회하고 읽어주는 비서입니다. "
        "sandbox 폴더 안의 파일만 다룰 수 있습니다. "
        f"sandbox 폴더의 경로는 {SANDBOX_PATH} 이며, 도구를 호출할
때는 이 절대 경로를 사용합니다."
    ),
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
```

```
                    command="npx",
                    args=["-y", "@modelcontextprotocol/server
-filesystem", SANDBOX_PATH],
                ),
                timeout=30,  # MCP 서버 시작 대기 시간(초). 기본값
5초
            ),
            tool_filter=["list_directory", "read_text_file"],
        )
    ],
)
```

```
# filesystem_agent/__init__.py
from . import agent
```

> **참고**
> `LlmAgent`는 지금까지 사용한 `Agent`와 같은 클래스입니다. ADK에서 `Agent`는 `LlmAgent`의 다른 이름이며, 둘 중 어느 것을 써도 동작은 같습니다.

> **참고**
> `timeout=30`은 ADK가 MCP 서버의 시작을 기다리는 시간입니다. 기본값은 5초이며, `npx`가 패키지를 내려받거나 PC가 느린 경우 5초 안에 서버가 준비되지 않아 연결 오류가 발생할 수 있습니다. `read_text_file`은 텍스트 파일을 읽는 도구입니다. 같은 기능의 `read_file`은 서버에서 사용 중단(deprecated) 예정으로 표시되어 있으므로 사용하지 않습니다.

Step 4. 실행 및 테스트

먼저 Filesystem MCP 서버 패키지를 미리 내려받습니다. `my-adk` 폴더에서 아래 명령을 실행하고, `Secure MCP Filesystem Server running on stdio`가 출력되면 `Ctrl+C`로 종료합니다. `일괄 작업을 끝내시겠습니까 (Y/N)?`가 표시되면 `Y`를 입력합니다. 이 과정을 생략하면 첫 실행 시 다운로드 시간 때문에 연결 시간 초과가 발생할 수 있습니다.

```
npx -y @modelcontextprotocol/server-filesystem sandbox
```

`my-adk` 폴더(프로젝트 루트)에서 가상환경이 활성화된 상태로 실행합니다.

```
adk web
```

브라우저(`http://127.0.0.1:8000/dev-ui/`)의 Agent 목록에서 `filesystem_agent`(폴더명)를 선택하고 아래 질문을 입력합니다.

- "sandbox 폴더에 어떤 파일이 있어?"
- "sample.txt 내용을 읽어줘."

> **관찰 포인트**: ADK 웹 UI의 Events 탭에서 에이전트가 `list_directory` → `read_text_file` 순으로 MCP 도구를 호출하는 과정이 그대로 보입니다. 이것이 2.6절에서 배운 데이터 흐름이 실제로 동작하는 모습입니다.

확인 문제 (실습 7-1)

1. `tool_filter=["list_directory", "read_text_file"]`를 지우고 다시 실행하면 어떤 변화가 생기는지 확인해보세요.
2. `SANDBOX_PATH` 밖에 있는 파일을 읽어달라고 요청하면 어떤 응답이 나오나요? 왜 그럴까요?

3.3 실습 7-2 — 나만의 MCP 서버 만들기 (FastMCP)

목표: MCP 서버를 직접 만들어 "Tool Provider" 역할을 이해합니다. 간단한 계산기·환율 조회 도구 두 개를 제공하는 서버를 만듭니다.

Step 1. 서버 파일 생성

```
my-adk/
└── my_mcp_server/
    └── server.py
```

Step 2. `server.py` 작성 — FastMCP로 도구 두 개 정의

```
# my_mcp_server/server.py
from fastmcp import FastMCP
import httpx

mcp = FastMCP("Lab Tools Server")

@mcp.tool()
def add_numbers(a: float, b: float) -> float:
    """두 숫자를 더합니다."""
    return a + b

@mcp.tool()
def get_exchange_rate(from_currency: str, to_currency: str) -
> dict:
    """두 통화 간의 환율을 조회합니다. 예: from_currency='USD', to_
```

```
currency='KRW'"""
    url = f"https://api.frankfurter.dev/v1/latest?from={from_
currency}&to={to_currency}"
    response = httpx.get(url, timeout=10)
    response.raise_for_status()
    data = response.json()
    return {
        "from": from_currency,
        "to": to_currency,
        "rate": data["rates"].get(to_currency),
        "date": data["date"],
    }

if __name__ == "__main__":
    mcp.run()  # 기본값: stdio 전송 방식
```

> **참고**
> `httpx`는 `fastmcp`를 설치할 때 함께 설치되는 HTTP 라이브러리입니다. Chapter 4의 `requests`와 같은 역할을 합니다.

Step 3. 서버 단독 실행으로 동작 검증

서버가 올바르게 도구를 노출하는지 ADK 없이 먼저 확인합니다. fastmcp와 함께 설치된 FastMCP CLI를 사용합니다.

```
fastmcp inspect my_mcp_server\server.py
```

서버 이름과 Tools 개수가 출력되면 정상입니다.

도구 목록과 설명까지 확인하려면 다음 명령을 사용합니다.

```
fastmcp list my_mcp_server\server.py
```

`add_numbers`, `get_exchange_rate` 두 도구가 docstring 설명과 함께 목록에 보이면 성공입니다.

> **주의**
> 공식 MCP SDK의 `mcp dev` 명령은 fastmcp 패키지로 만든 서버 객체와 호환되지 않아 "not a valid server object" 오류가 발생합니다. fastmcp로 만든 서버는 반드시 `fastmcp` CLI로 검증하세요.

확인 문제 (실습 7-2)

1. `add_numbers`처럼 두 수를 곱하는 `multiply_numbers` 도구를 추가해보세요.
2. 함수의 docstring을 지우고 `fastmcp list`로 다시 확인하면 무엇이 달라지나요? (LLM이 도구를 고를 때 docstring이 왜 중요한지 생각해보세요.)

3.4 실습 7-3 — 내 MCP 서버를 ADK 에이전트에 연동

목표: 3.3에서 만든 나만의 MCP 서버를 실제 ADK 에이전트와 연결해, "Root Agent → MCP Client → MCP Server → Tool" 전체 구조를 완성합니다.

Step 1. 에이전트 폴더 만들기

```
my-adk/
├── my_mcp_server/
│   └── server.py
└── lab_tools_agent/
    ├── __init__.py
    └── agent.py
```

Step 2. `agent.py` 작성

```
# lab_tools_agent/agent.py
import os
import sys
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import Std
ioConnectionParams
from mcp import StdioServerParameters

# my-adk/my_mcp_server/server.py의 절대 경로
SERVER_SCRIPT_PATH = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file_
_))),
    "my_mcp_server", "server.py",
)

root_agent = LlmAgent(
    model="gemini-3.1-flash-lite",
    name="lab_tools_agent",
    instruction=(
        "계산과 환율 조회를 도와주는 비서입니다. "
```

```
        "필요할 때 add_numbers, get_exchange_rate 도구를 사용하세
요."
    ),
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command=sys.executable,
                    args=[SERVER_SCRIPT_PATH],
                ),
                timeout=30,  # MCP 서버 시작 대기 시간(초). 기본값
5초
            ),
        )
    ],
)
```

```
# lab_tools_agent/__init__.py
from . import agent
```

> **참고**
> `command=sys.executable`은 "지금 adk web을 실행 중인 가상환경의 Python"을 그대로 사용하라는 뜻입니다. `"python3"`나 `"python"`처럼 이름으로 지정하면 Windows/macOS에 따라 명령이 다르거나, 가상환경 밖의 Python이 실행되어 `fastmcp`를 찾지 못하는 문제가 생길 수 있습니다.

Step 3. 실행 및 테스트

`my-adk` 폴더(프로젝트 루트)에서 가상환경이 활성화된 상태로 실행합니다.

```
adk web
```

Agent 목록에서 `lab_tools_agent`(폴더명)를 선택하고 다음을 질문해봅니다.

- "17 더하기 25는?"
- "지금 1 USD가 몇 KRW야?"

> **관찰 포인트**: 이번에는 `npx`로 실행되는 외부 패키지가 아니라, 우리가 직접 작성한 `server.py`가 별도 프로세스(Process, 실행 중인 프로그램 하나)로 실행되어 MCP 서버 역할을 합니다. 실습 7-1과 7-3을 비교하면 "기존 서버를 가져다 쓰는 것"과 "내가 서버를 만

> 들어 쓰는 것"의 차이가 코드 상으로는 `StdioServerParameters`의 `command`/`args`만 다르다는 점을 알 수 있습니다.

확인 문제 (실습 7-3)

1. 두 질문을 한 번에("17 더하기 25 하고, 그 결과를 USD에서 KRW로 환산해줘")처럼 물어보면 에이전트가 두 도구를 순서대로 호출하는지 확인해보세요.
2. `server.py`에 새로 추가한 `multiply_numbers` 도구를 에이전트가 잘 인식하는지 테스트해보세요. (서버 재시작이 필요할 수 있습니다.)

3.5 실습 7-4 — 원격/인증 MCP 서버 연동 (선택)

목표: 로컬 프로세스(stdio, Standard Input/Output: 같은 PC 안에서 프로그램끼리 표준 입출력로 데이터를 주고받는 방식)가 아니라, 네트워크 너머의 MCP 서버에 인증(Authentication, 호출자가 누구인지 확인) 헤더를 붙여 연결하는 방법을 익힙니다. 지금까지는 `StdioConnectionParams`를 썼다면, 이번에는 `StreamableHTTPConnectionParams`를 사용합니다. Streamable HTTP는 HTTP 위에서 요청을 보내고 응답을 스트리밍(Streaming, 생성되는 대로 조금씩 전송)으로 받는 MCP의 원격 전송 방식입니다.

코드 구조 비교

구분	실습 7-1 / 7-3 (로컬)	실습 7-4 (원격)
연결 클래스	`StdioConnectionParams`	`StreamableHTTPConnectionParams`
실행 주체	로컬 프로세스(`npx`, Python)	이미 배포되어 있는 원격 서버
인증	보통 불필요	API 키 / 토큰을 HTTP 헤더로 전달

Step 1. `agent.py` 작성

```
# remote_mcp_agent/agent.py
import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import Str
eamableHTTPConnectionParams

REMOTE_API_KEY = os.getenv("REMOTE_MCP_API_KEY", "")

root_agent = LlmAgent(
    model="gemini-3.1-flash-lite",
    name="remote_mcp_agent",
    instruction="원격 MCP 서버의 도구를 활용해 사용자 질문에 답하세
```

```
요.",
    tools=[
        McpToolset(
            connection_params=StreamableHTTPConnectionParams(
                url="https://your-remote-mcp-server.example.c
om/mcp",
                headers={
                    "Authorization": f"Bearer {REMOTE_API_KE
Y}",
                    "Content-Type": "application/json",
                },
            ),
        )
    ],
)
```

```
# remote_mcp_agent/__init__.py
from . import agent
```

Step 2. 나만의 서버를 원격처럼 실행해보기

강의실 네트워크나 별도 원격 서버가 없다면, FastMCP의 HTTP 모드로 로컬에서 "원격 서버 흉내"를 낼 수 있습니다.

실습 7-3의 `server.py`는 stdio 방식으로 계속 사용해야 하므로 수정하지 않고, HTTP 방식으로 실행하는 **별도 파일**을 만듭니다.

```
# my_mcp_server/server_http.py
from server import mcp

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", po
rt=8080)
```

VS Code에서 터미널을 하나 더 열고(터미널 패널의 **+** 버튼) 다음을 실행합니다. 새 터미널은 가상환경이 활성화되지 않은 상태로 열리므로 첫 줄에서 다시 활성화합니다. 이 터미널은 서버 역할을 하므로 닫지 않습니다.

```
venv\Scripts\activate
cd my_mcp_server
```

```
python server_http.py
```

서버를 종료할 때는 이 터미널에서 `Ctrl + C`를 누릅니다.

이제 `remote_mcp_agent/agent.py`의 `url`을 `http://127.0.0.1:8080/mcp`로 바꾸고, `headers`는 비워서(`{}`) 테스트해봅니다.

```
connection_params=StreamableHTTPConnectionParams(
    url="http://127.0.0.1:8080/mcp",
    headers={},
)
```

원래 터미널에서 `adk web`을 실행하고 `remote_mcp_agent`를 선택해 실습 7-3과 같은 질문을 해봅니다.

확인 문제 (실습 7-4)

1. `StdioConnectionParams`와 `StreamableHTTPConnectionParams`의 코드 구조상 차이는 무엇인가요?
2. 인증 헤더(`Authorization`)를 일부러 잘못 입력하면 어떤 오류가 발생하나요?

3.6 트러블슈팅 가이드

증상	원인	해결 방법
`NotImplementedError` (Windows)	asyncio subprocess 미지원	`adk web --no-reload`로 실행
MCP 연결 시간 초과 (`Timed out`, `Failed to create MCP session`)	MCP 서버가 5초(기본값) 안에 시작되지 않음. `npx` 첫 실행 시 패키지 다운로드 지연이 대표적 원인	`StdioConnectionParams`에 `timeout=30` 지정, `npx` 명령을 터미널에서 미리 한 번 실행
환율 조회 시 `302 Found` / `Redirect` 오류	이전 주소(`api.frankfurter.app`)가 새 주소로 이동됨. `httpx`는 기본적으로 리다이렉트를 따라가지 않음	URL을 `https://api.frankfurter.dev/v1/latest`로 수정
`'npx'은(는) 내부 또는 외부 명령... 이 아닙니다`	Node.js 미설치, 또는 설치 후 VS Code를 다시 실행하지 않음	Node.js 24 설치 후 VS Code 창을 모두 닫고 다시 실행
`이 시스템에서 스크립트를 실행할 수 없으므로...`	터미널이 명령 프롬프트가 아니라 PowerShell로 열려 있음	Chapter 2의 2.10절대로 기본 터미널을 Command Prompt로 변경

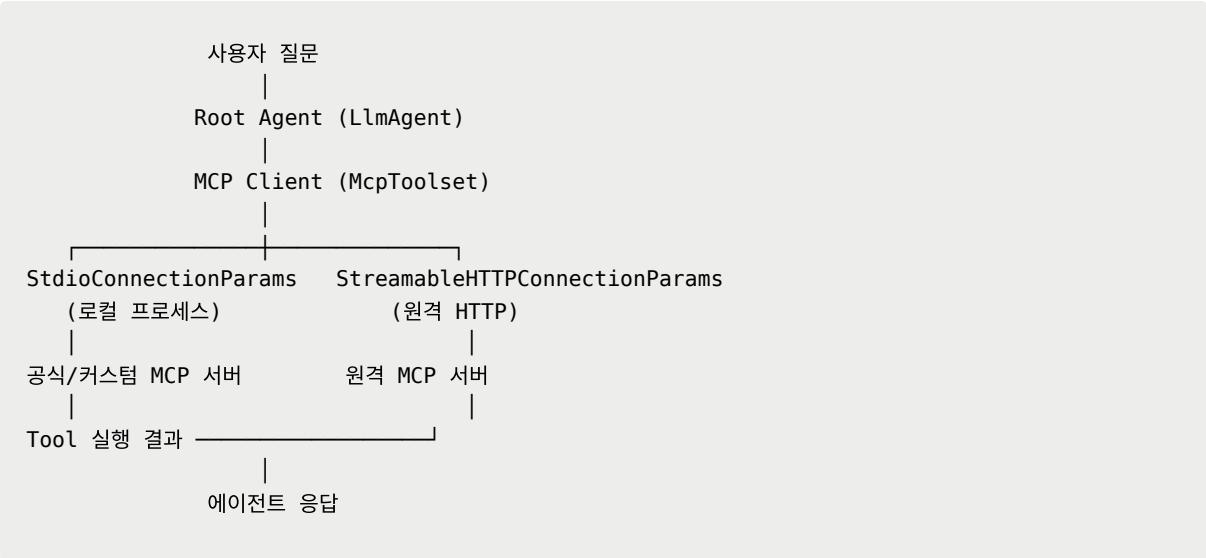
증상	원인	해결 방법
`adk` 또는 `fastmcp` 명령을 찾을 수 없음	새 터미널에서 가상환경을 활성화하지 않음	`my-adk` 폴더에서 `venv\Scripts\activate` 실행 후 `(venv)` 표시 확인
Agent 목록에 새 폴더가 보이지 않음	`__init__.py` 누락 또는 `adk web`을 다른 폴더에서 실행	`__init__.py` 확인, `my-adk` 루트에서 실행
에이전트가 도구를 전혀 호출하지 않음	`instruction`이 모호하거나 docstring 부재	instruction과 도구 docstring을 명확히 작성
`Connection closed` / 서버 응답 없음	서버 스크립트 경로 오류, 서버 자체 오류, `fastmcp` 미설치 Python 사용	`fastmcp list my_mcp_server\server.py`로 서버 단독 실행해 원인 분리
원격 서버 401/403 오류	인증 헤더 누락 또는 오타	`.env`의 API 키 값과 헤더 이름을 다시 확인
도구는 보이는데 실행 결과가 이상함	`tool_filter` 설정 실수, 캐시된 세션	세션을 새로 만들고 `tool_filter` 재확인

3.7 실습 정리 및 점검

오늘 완성한 것

- [x] ~~실습 7-1: 공식 Filesystem MCP 서버를 ADK 에이전트에 연동~~
- [x] ~~실습 7-2: FastMCP로 계산기·환율 도구를 제공하는 나만의 MCP 서버 제작~~
- [x] ~~실습 7-3: 내가 만든 MCP 서버를 ADK 에이전트와 연결해 end-to-end 흐름 완성~~
- [x] ~~실습 7-4: `StreamableHTTPConnectionParams`로 원격/인증 MCP 서버 연동 (선택)~~

구조 되짚어보기



1일차 전체 결과물

이번 장까지 마치면 `my-adk` 프로젝트에는 다음 Agent가 누적되어 있으며, `adk web` 목록에서 모두 선택할 수 있습니다.

폴더	만든 장	내용
`agent`	Chapter 2~5	시간·계산·주사위·환율·메모·날씨 Tool을 가진 멀티 Tool Agent
`helpdesk`	Chapter 6	Role·Format·Constraint를 설계한 사내 헬프데스크 Agent
`filesystem_agent`	Chapter 7	공식 Filesystem MCP 서버를 연결한 Agent
`lab_tools_agent`	Chapter 7	직접 만든 MCP 서버를 연결한 Agent

> **심화 학습 안내**: 이 과정에서는 다루지 않지만, 오늘 만든 커스텀 MCP 서버를 역방향으로 활용할 수도 있습니다. ADK 도구를 MCP 서버로 감싸 Claude Desktop 등 다른 MCP 클라이언트에서 사용하도록 노출하거나, 운영 환경(Cloud Run 등)에 배포하는 것도 가능합니다.

과제 (선택)

1. 실습 7-2의 서버에 도구를 하나 더 추가하고(예: 날씨 조회, 단위 변환 등), 실습 7-3의 에이전트가 새 도구를 잘 사용하는지 확인해 주세요.

체크포인트 질문

1. Chapter 3의 Tool Calling 방식과 비교했을 때, MCP가 해결하려는 문제는 무엇인가요? → **외부 시스템마다 Tool을 직접 구현하고 프로젝트마다 반복하는 문제. MCP는 표준 방식으로 이미 만들어진 서버를 재사용하게 함**
2. Host, MCP Client, MCP Server 중 "실제 기능(파일 읽기, DB 조회)을 수행하는 것"은 무엇인가요? → **MCP Server**
3. Google ADK에서 MCP Client 역할을 하는 구성 요소는 무엇인가요? → `McpToolset`
4. 실습 7-1과 7-3의 Agent 코드에서 달라진 부분은 어디인가요? → `StdioServerParameters`**의** `command`**와** `args`**(실행할 MCP 서버 프로그램)만 다르고 나머지 구조는 동일함**
5. 로컬 MCP 서버(stdio)와 원격 MCP 서버(HTTP)를 연결할 때 사용하는 클래스는 각각 무엇인가요? → `StdioConnectionParams`, `StreamableHTTPConnectionParams`